

Module 22: SQL Functions

SQL Functions are predefined or user-defined routines that accept input values, perform calculations or operations, and return a result. Functions help reduce repetitive code and make SQL queries more powerful and efficient.



Introduction

Consider a table:

EmployeeName	Salary
Kunal	50000
Rahul	65000

Suppose you need:

- Convert names to uppercase
- Calculate salary averages
- Extract year from a date
- Round numbers

Instead of writing complex logic manually, SQL provides:

Functions



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand SQL Functions
 - ✓ Use Built-in Functions
 - ✓ Use String Functions
 - ✓ Use Date Functions
 - ✓ Use Mathematical Functions
 - ✓ Create User-Defined Functions (UDFs)
 - ✓ Differentiate Functions and Procedures
-



Table of Contents

1. What are SQL Functions?
 2. Why Functions are Needed
 3. Types of SQL Functions
 4. String Functions
 5. Date Functions
 6. Mathematical Functions
 7. Aggregate Functions
 8. Conversion Functions
 9. User-Defined Functions (UDFs)
 10. Function vs Procedure
 11. Best Practices
 12. Interview Questions
 13. Business Analytics Examples
 14. Summary
 15. Practice Questions
-

1 What are SQL Functions?

Functions are reusable routines that perform specific operations and return a value.

Example

```
SELECT UPPER('kunal');
```

Output:

```
KUNAL
```

Structure

```
Function(Input)
```

↓

Process



Output

Why Functions Matter

Used in:

- Data Cleaning
 - Data Transformation
 - Reporting
 - Analytics
 - Dashboard Development
-

2 Why Functions are Needed

Without Functions:

Manual calculations
Repeated logic
Complex code

With Functions:

Simple queries
Cleaner code
Reusable logic

Example

```
SELECT AVG(Salary)
FROM Employees;
```

instead of manually calculating averages.

3 Types of SQL Functions

Type	Purpose
String Functions	Text manipulation
Date Functions	Date calculations
Mathematical Functions	Numeric operations
Aggregate Functions	Group calculations
Conversion Functions	Data type conversion
User-Defined Functions	Custom logic

4 String Functions

Used for text manipulation.

UPPER()

Converts text to uppercase.

```
SELECT UPPER('kunal');
```

Output:

```
KUNAL
```

LOWER()

Converts text to lowercase.

```
SELECT LOWER('KUNAL');
```

Output:

```
kunal
```

LEN()

Returns length of string.

```
SELECT LEN('Kunal');
```

Output:

```
5
```

CONCAT()

Combines strings.

```
SELECT CONCAT('Kunal', ' ', 'Yadav');
```

Output:

```
Kunal Yadav
```

LEFT()

Returns characters from left.

```
SELECT LEFT('Kunal',2);
```

Output:

```
Ku
```

RIGHT()

Returns characters from right.

```
SELECT RIGHT('Kunal',2);
```

Output:

```
al
```

SUBSTRING()

Extracts part of string.

```
SELECT SUBSTRING('Kunal',2,3);
```

Output:

```
una
```

REPLACE()

Replaces text.

```
SELECT REPLACE('SQL Server','Server','Database');
```

Output:

```
SQL Database
```

TRIM()

Removes spaces.

```
SELECT TRIM('  SQL ');
```

Output:

```
SQL
```

5 Date Functions

Used for working with dates.

GETDATE()

Current date and time.

```
SELECT GETDATE();
```

YEAR()

Extract year.

```
SELECT YEAR(GETDATE());
```

Output:

```
2026
```

MONTH()

Extract month.

```
SELECT MONTH(GETDATE());
```

DAY()

Extract day.

```
SELECT DAY(GETDATE());
```

DATEADD()

Add time interval.

```
SELECT DATEADD(YEAR,1,GETDATE());
```

DATEDIFF()

Difference between dates.

```
SELECT DATEDIFF(DAY,  
                '2025-01-01',  
                '2025-01-10');
```

Output:

```
9
```

EOMONTH()

Returns last day of month.

```
SELECT EOMONTH(GETDATE());
```

Business Uses

- Employee tenure
 - Monthly reports
 - Sales trends
-

Mathematical Functions

Used for calculations.

ABS()

Absolute value.

```
SELECT ABS(-100);
```

Output:

```
100
```

ROUND()

Rounds number.

```
SELECT ROUND(123.456,2);
```

Output:

```
123.46
```

CEILING()

Rounds up.

```
SELECT CEILING(12.1);
```

Output:

```
13
```

FLOOR()

Rounds down.

```
SELECT FLOOR(12.9);
```

Output:

```
12
```

POWER()

Exponent.

```
SELECT POWER(2,3);
```

Output:

```
8
```

SQRT()

Square root.

```
SELECT SQRT(64);
```

Output:

8

RAND()

Random number.

```
SELECT RAND();
```

7 Aggregate Functions

Work on multiple rows.

COUNT()

```
SELECT COUNT(*)  
FROM Employees;
```

SUM()

```
SELECT SUM(Salary)  
FROM Employees;
```

AVG()

```
SELECT AVG(Salary)  
FROM Employees;
```

MIN()

```
SELECT MIN(Salary)
FROM Employees;
```

MAX()

```
SELECT MAX(Salary)
FROM Employees;
```

Commonly Used With

```
GROUP BY
HAVING
```

Conversion Functions

Convert data types.

CAST()

```
SELECT CAST(100.75 AS INT);
```

Output:

```
100
```

CONVERT()

```
SELECT CONVERT(VARCHAR,  
               GETDATE(),  
               103);
```

Output

```
31/05/2026
```

TRY_CAST()

Returns NULL if conversion fails.

```
SELECT TRY_CAST('ABC' AS INT);
```

Output:

```
NULL
```

Why Useful?

Data Cleaning.



User-Defined Functions (UDFs)

Create custom functions.

Scalar Function

Returns single value.

Example

```
CREATE FUNCTION GetBonus
(
    @Salary DECIMAL(10,2)
)

RETURNS DECIMAL(10,2)

AS
BEGIN

    RETURN @Salary * 0.10;

END;
```

Execute

```
SELECT dbo.GetBonus(50000);
```

Output

```
5000
```

Table-Valued Function

Returns a table.

Example

```
CREATE FUNCTION GetEmployees()
RETURNS TABLE
AS

RETURN

(
    SELECT *
    FROM Employees
);
```

Execute

```
SELECT *  
FROM dbo.GetEmployees();
```

Uses

- Reusable calculations
 - Business rules
 - Reporting
-

10 Function vs Procedure

Important Interview Topic.

Function

Returns value
Can be used in SELECT
Cannot modify database directly (generally)

Procedure

Can execute multiple operations
Can modify database
Executed using EXEC

Comparison

Feature	Function	Procedure
Returns Value	Yes	Optional
Used in SELECT	Yes	No
EXEC Required	No	Yes

Feature	Function	Procedure
Modify Data	Limited	Yes

Example Function

```
SELECT dbo.GetBonus(50000);
```

Example Procedure

```
EXEC GetEmployees;
```

Best Practices

Use Built-in Functions First

Usually faster.

Avoid Functions in WHERE on Indexed Columns

Bad:

```
WHERE YEAR(OrderDate)=2025
```

Better:

```
WHERE OrderDate >= '2025-01-01'  
AND OrderDate < '2026-01-01'
```

Keep UDFs Simple

Complex functions may affect performance.

Use Meaningful Names

Good:

```
CalculateBonus  
GetEmployeeAge
```

Validate Data Types

Avoid conversion errors.

1 2 Common Interview Questions

What is a Function?

Reusable routine that returns a value.

Difference Between Function and Procedure?

Function returns values and can be used in SELECT.

Procedure is executed using EXEC.

What is a Scalar Function?

Returns single value.

What is a Table-Valued Function?

Returns a table.

What are Aggregate Functions?

Functions operating on multiple rows.

What is CAST?

Converts one data type to another.

1 3 Business Analytics Examples

Employee Name Standardization

```
SELECT UPPER(EmployeeName)
FROM Employees;
```

Monthly Sales Report

```
SELECT MONTH(OrderDate),
        SUM(SalesAmount)
FROM Orders
GROUP BY MONTH(OrderDate);
```

Customer Age

```
SELECT DATEDIFF(YEAR,
                BirthDate,
                GETDATE())
FROM Customers;
```

Salary Bonus

```
SELECT Salary,
        Salary * 0.10
FROM Employees;
```

Data Cleaning

```
SELECT TRIM(CustomerName)
FROM Customers;
```

KPI Dashboard

```
SELECT COUNT(*) AS Employees,
       AVG(Salary) AS AvgSalary,
       MAX(Salary) AS HighestSalary
FROM Employees;
```



Summary

In this module, you learned:

- ✓ String Functions
 - ✓ Date Functions
 - ✓ Mathematical Functions
 - ✓ Aggregate Functions
 - ✓ Conversion Functions
 - ✓ User-Defined Functions
 - ✓ Scalar Functions
 - ✓ Table-Valued Functions
 - ✓ Function vs Procedure
-



Practice Questions

Theory

1. What is a SQL Function?
2. Why are Functions used?
3. What are String Functions?
4. What are Date Functions?
5. What are Aggregate Functions?
6. What is CAST?

7. What is a Scalar Function?
 8. What is a Table-Valued Function?
 9. Difference between Function and Procedure?
 10. What are best practices for Functions?
-

Practical Exercises

Task 1

Convert employee names to uppercase.

Task 2

Extract current year.

Task 3

Calculate date difference.

Task 4

Round salary values.

Task 5

Create a bonus function.

Task 6

Create a table-valued function.

Challenge Project

Create:

```
Employees  
Customers  
Orders  
Sales
```

Build:

- Name Formatting Functions
- Bonus Calculation Function

- Customer Age Function
- Sales Analysis Queries
- KPI Dashboard Queries

using SQL Functions.



Next Module

→ **Module 23: Common Table Expressions (CTEs)**

Topics Covered:

- What is a CTE?
- CTE Syntax
- Multiple CTEs
- Recursive CTEs
- CTE vs Subquery
- CTE vs Temp Table
- Analytics Use Cases
- Interview Questions